Join our community, grow your knowledge and learn from others!
Sign In
Don't have an Epic Games account? Sign up

Managing Game Code

Add code to your project and generate project files for IDEs.

Choose your operating system

Windows · macOS · Linux

User Project Files

Unreal Engine (UE) recognizes two different types of projects: native and foreign.

Native Projects

Native projects are projects that exist within the same root directory as the engine that was used to create them. As a result, they have the following directory structure:

```
.
├── MyUE5/
│   ├── Engine/
│   │   ├── Binaries/
│   │   ├── Build/
│   │   ├── Config/
│   │   └── ...
│   ├── ...
│   ├── PROJECT1/
│   │   ├── Binaries/
│   │   ├── Build/
│   │   ├── Config/
│   │   ├── Content/
│   │   ├── PROJECT1.uproject
│   ├── PROJECT2/
│   │   ├── Binaries/
│   │   ├── Build/
│   │   ├── Config/
│   │   ├── Content/
│   │   ├── PROJECT2.uproject
│   ├── ...
│   ├── Default.uprojectdirs
│   ├── GenerateProjectFiles.bat
│   ├── ...
└── ...
```

Copy full snippet

With this structure `PROJECT1` and `PROJECT2` are both native projects. Projects with this directory structure do not require any extra configuration in order to determine which editor to use, these projects will automatically detect the editor that is built within the root directory. This is true whether you are using a source build or a binary build of UE.

Projects can be put into other root folders, or any subfolder that has been defined in `Default.uprojectdirs` (for example, `MOUT/Games`). The `.uprojectdirs` file is a helper file that directs the engine to discover projects in subdirectories of `MOUT`. You can have multiple `.uprojectdirs` files in the same `MOUT` directory. These files are not required to be named `Default.uprojectdirs`, they can have any name you wish as long as the file extension is `.uprojectdirs`.

Foreign Projects

Foreign projects are those where your project directory is in some location other than under the same root directory as the engine directory. An example of this would be:

```
.
├── UE/
│   ├── Engine/
│   │   ├── Binaries
│   │   ├── Build
│   │   ├── Config
│   │   ├── ...
│   ├── ...
│   ├── Default.uprojectdirs
│   ├── GenerateProjectFiles.bat
│   ├── ...
├── PROJECT1/
│   ├── Binaries
│   ├── Build
│   ├── Config
│   ├── ...
│   ├── PROJECT1.uproject
├── PROJECT2/
│   ├── Binaries
│   ├── Build
│   ├── Config
│   ├── ...
│   ├── PROJECT2.uproject
└── ...
```

Copy full snippet

When a foreign project is created from a launcher (binary) build, the `EngineAssociation` field in its `.uproject` file is set to the version number the launcher engine that was used to create it. In case you created the project from a source-built version of the Unreal Editor it will put in the **Globally Unique Identifier (GUID)** of that editor. This GUID is automatically generated and saved on your computer the first time you launch your source-built editor. The locations where these are stored are different operating systems are:

- **Windows**
  - Registry `Computer\HKEY_CURRENT_USER\SOFTWARE\Epic Games\Unreal Engine\Builds`
- **Mac**
  - File `~/Library/Application Support/Epic/UnrealEngine/Install.ini`
- **Linux**
  - File `~/.config/Epic/UnrealEngine/Install.ini`

For example, say `PROJECT1` was created from a source-built version of the Unreal Editor, then the `PROJECT1.uproject` file contains something of the form:

```
{
  "FileVersion": 3,
  "EngineAssociation": "{IFF8386A-EA6F-48EC-859E-6CBB21BF15A0}",
  "Category": "",
  "Description": "",
  "Modules": [
    ...
  ],
  "Plugins": [
    ...
  ],
  "TargetPlatforms": [
    ...
  ]
}
```

Copy full snippet

where `IFF8386A-EA6F-48EC-859E-6CBB21BF15A0` is the GUID associated with the source-built version of the Unreal Editor used to create the project. In contrast, suppose `PROJECT1` was created from a UE 5.1 launcher version of the Unreal Editor, then the `PROJECT1.uproject` file contains:

```
{
  "FileVersion": 3,
  "EngineAssociation": "5.1",
  "Category": "",
  "Description": "",
  "Modules": [
    ...
  ],
  "Plugins": [
    ...
  ],
  "TargetPlatforms": [
    ...
  ]
}
```

Copy full snippet

Change a Project's Unreal Engine Version

You can change the Unreal Engine version that a project is used to open a project by first right clicking on the project's `.uproject` file and choosing **Switch Unreal Engine version...**. This will bring up a dialogue box with a dropdown menu where you can select the version of UE with which to open this project. When this dialogue box first appears, it will display the current Unreal Engine distribution that is set to open this project. Launcher builds in this dropdown menu are identified by their version number (for example, 4.27 or 5.0) and source builds are identified as **Binary build** or **Source build** followed by the build's directory. This directory is the value associated with the GUID key associated with the `EngineAssociation` field in the `.uproject` file. To switch the engine version, select your desired Unreal Engine distribution from the dropdown menu. The system will then regenerate project files to implement this change.

Adding Code to Projects

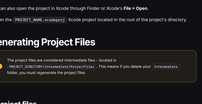
C++ Class Wizard

The **C++ Class Wizard** provides a fast and easy way to add native C++ code classes into your project for you to extend with your own functionality, if you wish. This converts a content-only project into a code project. You can access the C++ Class Wizard and create a new C++ class by following these steps:

> Check Setting Up Visual Studio to ensure that you have a compatible version of Visual Studio installed before proceeding. For information about the correct version of Xcode, refer to the Unreal Engine Release Notes.

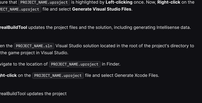
1. In the main editor, select **Tools > New C++ Class...**.

2. The **C++ Class Wizard** will appear and show **Common Classes** by default. If you do not see the class you are looking for, you can view the entire Class hierarchy listing by selecting **All Classes**.

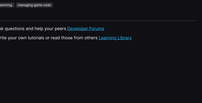


Common Classes · All Classes

3. Choose the Class you want to add. For the purposes of this demonstration, we will choose to create a new **Actor** Class. Select the **Actor** Class, then click **Next >**.



4. You will then be prompted to enter a **Name** for your new Class. Do so, then click **Create Class**. This will create the header (`.h`) and source (`.cpp`) files.



> Class names can only contain alphanumeric characters and cannot contain spaces. The field will notify you if you enter an invalid name.

5. In Unreal Engine, **Live Coding** is now enabled by default. A Live Coding window will appear and compile the new class files that were created.

6. The code will immediately open in Visual Studio, ready for editing.

   The code will immediately open in Xcode, ready for editing.

Development Environment

Code files can be created through **Visual Studio** and added to the game project through the **Solution Explorer** in the usual manner. You can also add code files to the appropriate folders outside of Visual Studio and rebuild the solution and project files automatically. This makes it easy to add lots of files quickly through the operating system UI, and also makes working on teams easier as the solution and project files do not need to be synced between members of the team. Each person can just sync the code files and rebuild the project files locally.

Code files can be created through **Xcode** and added to the game project through the **Project Navigator** in the usual manner. You can also add code files to the appropriate folders outside of Xcode and rebuild the project file automatically. This makes it easy to add lots of files quickly through the operating system UI, and also makes working on teams easier as the solution and project files do not need to be synced between members of the team. Each person can just sync the code files and rebuild the project files locally.

Opening Projects in the Development Environment

If your project is already open in the **Unreal Editor**, you can easily open it in Visual Studio by selecting **Open Visual Studio** from the **Tools** menu.

You can also open the project in Visual Studio through Windows Explorer or Visual Studio's **File > Open > Project/Solution**.

If your project is inside the UE root directory:

- Open the `UE5.sln` Visual Studio solution located in the UE root directory.

If your project is outside of the UE root directory:

- Open the `PROJECT_NAME.sln` Visual Studio solution located in the root of the project's directory.

If your project is already open in the editor, you can easily open it in Xcode by selecting **Open in Xcode** from the **File** menu.

You can also open the project in Xcode through Finder or Xcode's **File > Open**.

- Open the `PROJECT_NAME.xcodeproj` Xcode project located in the root of the project's directory.

Generating Project Files

> This project files are considered intermediate files - located in `PROJECT_DIRECTORY/Intermediate/ProjectFiles`. This means if you delete your `Intermediate` folder, you must regenerate the project files.

.uproject files

1. Navigate to the location of `PROJECT_NAME.uproject` in Windows Explorer.

2. Ensure that `PROJECT_NAME.uproject` is highlighted by **Left-clicking** once. Now, **Right-click** on the `PROJECT_NAME.uproject` file and select **Generate Visual Studio Files**.

3. **UnrealBuildTool** updates the project files and the solution, including generating Intellisense data.

1. Open the `PROJECT_NAME.sln` Visual Studio solution located in the root of the project's directory to view the game project in Visual Studio.

1. Navigate to the location of `PROJECT_NAME.uproject` in Finder.

2. **Right-click** on the `PROJECT_NAME.uproject` file and select Generate Xcode Files.

1. UnrealBuildTool updates the project

1. Open the `PROJECT_NAME.xcodeproj` Xcode project located in the root of the project's directory to view the game project in Xcode.

programming · managing game code